

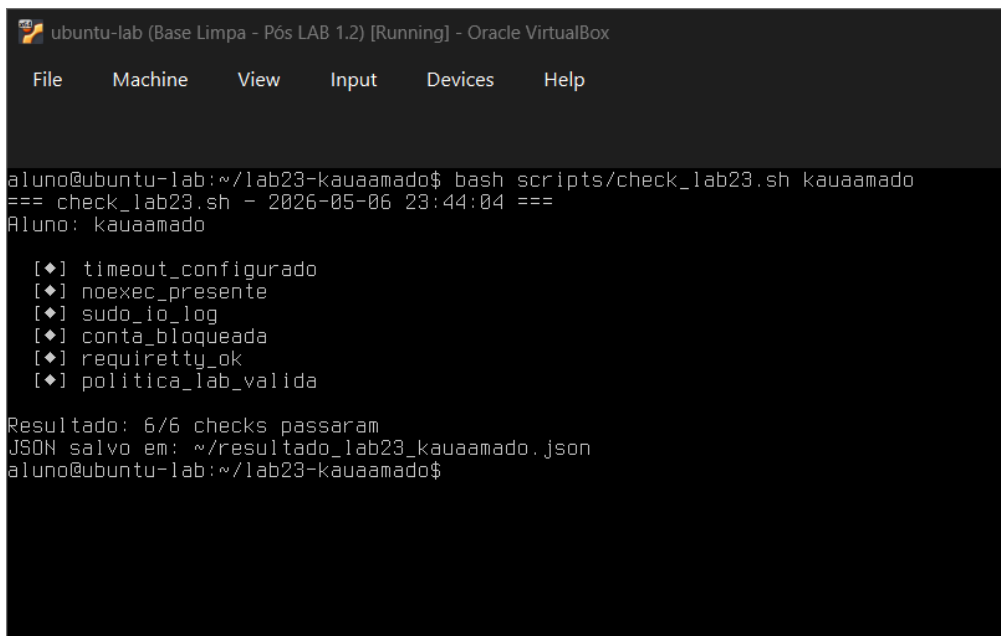
Relatório de Conclusão de Laboratório

LAB 2.3 — Hardening Avançado do Sudo

Campo	Preencher
Nome completo	Kauã Willians Lopes Amado
Matrícula	202511920015
Data de entrega	6/5/2026
Versão	1.0

Seção A — Confirmação da Entrega Automática

Cole abaixo um screenshot do terminal mostrando a execução do script `check_lab23.sh` e a mensagem de resultado final (ex.: 6/6 checks passaram – JSON gerado). Prova que o aluno executou o script pessoalmente e não apenas copiou o arquivo JSON.



```
ubuntu-lab (Base Limpa - Pós LAB 1.2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

aluno@ubuntu-lab:~/lab23-kauaamado$ bash scripts/check_lab23.sh kauaamado
=== check_lab23.sh - 2026-05-06 23:44:04 ===
Aluno: kauaamado

[+] timeout_configurado
[+] noexec_presente
[+] sudo_io_log
[+] conta_bloqueada
[+] requiretty_ok
[+] politica_lab_valida

Resultado: 6/6 checks passaram
JSON salvo em: ~/resultado_lab23_kauaamado.json
aluno@ubuntu-lab:~/lab23-kauaamado$
```

Seção B — Evidências Visuais sem Correspondente no Script (Canal 2a)

Screenshots de itens verificados manualmente que o script não consegue capturar automaticamente.

B.1 — Dois comandos sudo consecutivos pedindo senha separadamente

Terminal mostrando `sudo whoami` e `sudo id` executados um após o outro, ambos pedindo senha (confirma `timestamp_timeout=0` ou valor baixo).

```
aluno@ubuntu-lab:~$ sudo whoami
[sudo] password for aluno:
root
aluno@ubuntu-lab:~$ sudo id
uid=0(root) gid=0(root) groups=0(root)
aluno@ubuntu-lab:~$
```

B.2 — Shell escape bloqueado pelo NOEXEC dentro do vim

Terminal mostrando a tentativa de escapar para shell dentro do vim com `:!bash` sendo bloqueada pela diretiva `NOEXEC` (mensagem de erro do vim ou shell não iniciado).

[IMG_noexec_bloqueado — screenshot da tentativa de shell escape dentro do vim com NOEXEC ativo]

B.3 — Comandos perigosos bloqueados para o operador

Terminal mostrando tentativas do operador de executar `sudo rm /etc/hosts`, `sudo /bin/bash` e `sudo passwd root`, todas retornando mensagem de bloqueio.

```
operador@ubuntu-lab:~$ sudo passwd root
[sudo] password for operador:
Sorry, user operador is not allowed to execute '/usr/bin/passwd root' as root on ubuntu-la
operador@ubuntu-lab:~$ sudo rm /etc/hosts
[sudo] password for operador:
Sorry, user operador is not allowed to execute '/usr/bin/rm /etc/hosts' as root on ubuntu-
operador@ubuntu-lab:~$
```

B.4 — Replay de sessão com sudoreplay (bônus)

Terminal mostrando a execução de `sudo sudoreplay /var/log/sudo-io/...` reproduzindo uma sessão gravada.

```
aluno@ubuntu-lab:~$ sudo sudoreplay /var/log/sudo-io/sudo-io/aluno/00/00/01
Replaying sudo session: /usr/bin/ls /root
vboxpostinstall.sh
aluno@ubuntu-lab:~$
```

Seção C — Dupla Evidência: Screenshots com Check_ID (Canal 2b)

Para cada `check_id` abaixo, cole um screenshot do terminal mostrando o comando executado e o resultado esperado.

check_timeout_configurado {#}

Campo	Valor
Comando	sudo grep "timestamp_timeout" /etc/sudoers.d/politica_lab
Esperado	Linha Defaults timestamp_timeout=0 (ou valor <= 5) visível no arquivo de política

```
aluno@ubuntu-lab:~$ sudo grep "timestamp_timeout" /etc/sudoers.d/politica_lab
Defaults timestamp_timeout=5
aluno@ubuntu-lab:~$ _
```

check_noexec_presente {#}

Campo	Valor
Comando	sudo grep "NOEXEC" /etc/sudoers.d/politica_lab
Esperado	Linha com NOEXEC: referenciando o alias de editores (ex.: operador ALL=(ALL) NOEXEC: EDITORES)

```
aluno@ubuntu-lab:~$ sudo grep "NOEXEC" /etc/sudoers.d/politica_lab
operador ALL=(ALL) NOEXEC: SERVICOS, REDE, !PERIGOSOS
aluno@ubuntu-lab:~$ _
```

check_sudo_io_log {#}

Campo	Valor
Comando	sudo ls /var/log/sudo-io/aluno/ (ou com o seu nome de usuário)
Esperado	Pelo menos um diretório com ID numérico de sessão listado (confirma que

Campo

Valor

o logging de I/O está
ativo e gravando)

```
aluno@ubuntu-lab:~$ sudo ls /var/log/sudo-io/sudo-io/aluno/
00 seq
aluno@ubuntu-lab:~$ _
```

check_conta_bloqueada {#}

Campo

Valor

Comando

sudo passwd -S
games e getent
passwd games

Esperado

passwd -S retorna
games L ... (L =
locked); getent
mostra
/usr/sbin/nologin
ou /sbin/nologin no
campo de shell

```
aluno@ubuntu-lab:~$ sudo passwd -S games
games L 2026-02-10 0 99999 7 -1
aluno@ubuntu-lab:~$ _
```

check_requiretty_ok {#}

Campo

Valor

Comando

sudo grep
"requiretty"
/etc/sudoers.d/politi
tica_lab

Esperado

Linha Defaults
requiretty visível no
arquivo de política

```
aluno@ubuntu-lab:~$ sudo grep "requiretty" /etc/sudoers.d/politica_lab
Defaults requiretty
aluno@ubuntu-lab:~$
```

check_politica_lab_valida {#}

Campo

Valor

Comando

sudo visudo -c -f
/etc/sudoers.d/politica_lab

Esperado

Mensagem
/etc/sudoers.d/politica_lab: parsed OK
— confirma que o
arquivo não tem
erros de sintaxe

```
aluno@ubuntu-lab:~$ sudo visudo -c -f /etc/sudoers.d/politica_lab
/etc/sudoers.d/politica_lab: parsed OK
aluno@ubuntu-lab:~$
```

Seção D — Respostas às Perguntas de Fixação

Responda em até 4 linhas cada.

1. O que é um “shell escape” em editores como vim e less? Como o NOEXEC previne esse tipo de ataque?

Resposta: O shell escape ocorre quando um editor permite executar comandos do terminal a partir dele (ex: `!bash`). Se executado via sudo, o shell resultante obtém privilégios totais de root. A diretiva NOEXEC previne isso bloqueando a execução de novos processos, impedindo que o editor chame o shell.

2. Qual a diferença entre `sudo vim arquivo` e `sudoedit arquivo`? Por que `sudoedit` é mais seguro?

Resposta: O `sudo vim` executa o editor diretamente como root, abrindo brechas para escapes. O `sudoedit` cria uma cópia temporária do arquivo, permitindo a edição sem privilégios elevados, e só aplica as mudanças como root após o fechamento. Ele é mais seguro porque falhas ou escapes ocorrerão apenas no nível do usuário comum.

3. Por que o campo `requiretty` no Defaults aumenta a segurança? O que acontece se um script malicioso tentar executar sudo sem terminal?

Resposta: **`requiretty`** exige que o sudo seja invocado a partir de uma sessão de terminal legítima. Se um script malicioso tentar rodar o sudo em segundo

plano (como via vulnerabilidade web), o sistema negará a execução. Isso evita ataques remotos e automatizados de escalada de privilégio.

4. Por que manter contas de sistema com shell de login (/bin/bash) é um risco, mesmo que não tenham senha definida?

Resposta: Ter um shell válido permite que um invasor no sistema mude para essa conta via su ou sudo e obtenha um terminal. Como contas de sistema operam serviços automáticos e não precisam de interação antrópica, elas devem usar /usr/sbin/nologin. Isso garante seu funcionamento sem abrir um vetor de ataque.

5. (Desafio) Um auditor encontrou: devops ALL=(ALL) ALL, !PERIGOSOS. Ele afirma que pode ser contornado. Como? O que você faria para corrigir?

Resposta: O bloqueio pode ser contornado porque o usuário pode copiar os binários proibidos para outro local ou usar comandos não listados que possuem funções de shell. A correção ideal é seguir o princípio do menor privilégio: remover o acesso irrestrito (ALL) e definir uma lista explícita com apenas os comandos necessários.

A.7 — Checklist de Submissão — Verificação Final do Aluno

Marque todos os itens antes de executar o `git push`:

- ☒ O script `check_lab23.sh` foi executado e exibiu o resultado no terminal
- ☒ O arquivo `resultado_lab23_<meu_nome>.json` existe na pasta `~/`
- ☐ Todos os placeholders `[IMG_` do template foram preenchidos
- ☒ Todos os screenshots estão legíveis e mostram claramente o resultado esperado
- ☒ O PDF foi gerado com o nome correto: `lab23_relatorio_<meu_nome>.pdf`
- ☒ Os dois arquivos (JSON e PDF) estão na pasta correta do repositório
- ☒ `git add` foi executado para os dois arquivos
- ☒ `git commit` foi executado com mensagem padrão: `"lab23: entrega <meu_nome>"`
- ☒ `git push` foi executado sem erros
- ☒ `git log --oneline` confirma que o commit aparece no histórico
- ☒ A entrega está dentro do prazo definido pelo professor